

CodeColosseum

Testing Document

Kerem Adalı (150123055) Buğra Kaya (150123045)
Ali Talip Keleş (150122029)

Course: CSE3044 Software Engineering – Spring 2025–2026

Standard: IEEE Std 829 (Software Test Documentation)

Date: May 29, 2026

Contents

1	Test Plan	3
1.1	Purpose	3
1.2	Scope	3
1.2.1	Features Tested	3
1.2.2	Features NOT Tested (and Why)	3
1.3	Test Environment	4
1.4	Pass/Fail Criteria	5
2	Test Case Specification	5
2.1	JwtUtil – Token Generation and Validation	5
2.2	MatchmakingDto – Elo Tolerance Algorithm (<code>canMatchWith</code>)	5
2.3	MatchmakingService – Difficulty Assignment (<code>calculateDifficulty</code>)	6
2.4	RatingService – Elo Calculation (<code>calculateRatingChange</code>)	6
2.5	ProblemController – HTTP Authorization Boundaries	7
2.6	JudgeService – Code Execution and Verdict Evaluation	7
3	Test Execution	8
3.1	Test Files and Configurations	8
3.2	Execution Commands	8
3.3	Execution Log (May 29, 2026)	9
4	Bug Reports	10
5	Test Results	11
6	Evaluation	12
6.1	Coverage Analysis	12
6.2	What the Tests Do Not Cover	13

7	Test Summary Report	13
8	Decision Log – Testing Strategy	14
8.1	DS-01 – Use Mockito for repository isolation rather than a test database .	14
8.2	DS-02 – Exclude WebSocket handlers from automated tests	14
8.3	DS-03 – Use MockMvc <code>standaloneSetup</code> instead of <code>@WebMvcTest</code> for the controller integration test	15

1 Test Plan

1.1 Purpose

This document describes the testing activities performed to verify that the core business logic of CodeColosseum behaves according to the Software Requirements Specification (SRS) and the Design Specification Document (DSD). It is produced in accordance with IEEE Std 829.

1.2 Scope

1.2.1 Features Tested

Feature	SRS Reference	Test Suite
JWT token generation and validation	FR-1 (Authentication)	JwtUtilTest
Elo-based matchmaking tolerance algorithm	FR-2 (Matchmaking)	MatchmakingTest
Problem difficulty assignment from average Elo	FR-2 (Matchmaking)	MatchmakingTest
Elo rating recalculation (win / loss / draw / floor)	FR-5 (Rating System)	RatingServiceTest
HTTP authorization boundary – missing token → 401	FR-1 (Auth)	ProblemControllerTest
HTTP authorization boundary – USER role on admin endpoint → 403	FR-6 (Problem Approval)	ProblemControllerTest
HTTP delegation – valid token reaches service → 200	FR-6 (Problem Approval)	ProblemControllerTest
Code execution and verdict evaluation	FR-4 (Automated Code Evaluation)	JudgeServiceTest

1.2.2 Features NOT Tested (and Why)

Feature	Reason Not Tested
WebSocket handlers (MatchHandler, MatchmakingHandler)	Require live WebSocket sessions with real timing. The concurrency and network behavior is verified by manual end-to-end demonstration during the project presentation.

Feature	Reason Not Tested
Judge0 API live integration	The HTTP request/response communication with a live Judge0 instance is excluded from automated testing. The internal mapping and compilation/verdict processing logic inside <code>JudgeService</code> itself is fully unit-tested in <code>JudgeServiceTest</code> using mocked responses.
React frontend	No automated UI test harness is in place. Frontend correctness was validated through manual browser testing throughout development.
Database SQL correctness (repository layer)	Repositories use PostgreSQL-specific SQL. Running them requires a live PostgreSQL instance. Mocking repositories in unit tests deliberately excludes SQL from scope; a test-container integration test would be the correct fix in a production project.
<code>LeaderboardService</code>	Returns the top 10 users ordered by Elo – a single <code>ORDER BY</code> query with no conditional logic. No business rules to verify beyond the SQL itself (excluded above).

Three levels of testing are applied:

Unit testing (white-box): Each class is tested in isolation. Repository and service dependencies are replaced with Mockito mocks, so no database or external service is required. The focus is on conditional logic, boundary values, and mathematical invariants.

Integration testing (HTTP layer slice): `ProblemControllerTest` uses Spring's `MockMvc` in `standaloneSetup` mode to exercise the full request-dispatch-response cycle of the controller without starting a real HTTP server or loading the full Spring application context. This verifies that the controller correctly enforces authentication and role checks at the HTTP boundary.

Static Code Analysis (static testing): ESLint static analysis runs over the React frontend codebase to verify syntax, style conventions, and catch hooks-related dependency errors at compile-time.

1.3 Test Environment

- Java 21
- Spring Boot 4.0.3
- JUnit 5 (Jupiter)
- Mockito (via `spring-boot-starter-data-jdbc-test`)
- Spring Test / MockMvc (via `spring-boot-starter-webmvc-test`)
- No external database or network required for any automated test

1.4 Pass/Fail Criteria

A test **passes** if the actual output exactly matches the expected output specified in Section 2. A test **fails** if the output differs, an exception is thrown unexpectedly, or the build does not compile. The suite is considered acceptable if all tests pass (**Failures: 0, Errors: 0**).

2 Test Case Specification

2.1 JwtUtil – Token Generation and Validation

ID	Description	Input	Expected Output
TC-JWT-01	Token carries username claim	<code>generateToken(uuid, "alice", "USER")</code>	<code>claims.get("username") == "alice"</code>
TC-JWT-02	Token carries role claim	<code>generateToken(uuid, "alice", "ADMIN")</code>	<code>claims.get("role") == "ADMIN"</code>
TC-JWT-03	Token subject is the user UUID	<code>generateToken(id, "alice", "USER")</code>	<code>claims.getSubject() == id.toString()</code>
TC-JWT-04	Tampered signature is rejected	Valid token with last 4 chars replaced by "XXXX"	<code>validateToken(tampered)</code> returns null
TC-JWT-05	Completely invalid string is rejected	"this.is.not.a.jwt"	<code>validateToken(...)</code> returns null

2.2 MatchmakingDto – Elo Tolerance Algorithm (canMatchWith)

Initial tolerance for a freshly queued player = **50**. Tolerance increases by 50 per completed 30-second interval: $tolerance = 50 + \lfloor waitSeconds/30 \rfloor \times 50$. Effective tolerance = $\min(tolerance_{p1}, tolerance_{p2})$.

ID	Description	Input	Expected
TC-MM-01	Identical Elo always matches	p1=1000, p2=1000, both fresh	true
TC-MM-02	Diff exactly at tolerance (50) – allowed (boundary)	p1=1050, p2=1000, both fresh	true
TC-MM-03	Diff one above tolerance (51) – rejected (boundary)	p1=1051, p2=1000, both fresh	false
TC-MM-04	Both waited 35 s – tolerance=100, diff=100 accepted	p1=1100, p2=1000, both 35 s wait	true

ID	Description	Input	Expected
TC-MM-05	Long-waiting player cannot override fresh opponent's tolerance	p1=1070 (35 s wait), p2=1000 (fresh)	false

2.3 MatchmakingService – Difficulty Assignment (calculateDifficulty)

Boundaries: EASY $\leq$ 1200 < MEDIUM $\leq$ 1500 < HARD

ID	Description	Input (elo1, elo2)	Expected
TC-DIFF-01a	Average below boundary (avg=1000)	1000, 1000	"EASY"
TC-DIFF-01b	Average exactly at EASY boundary (avg=1200)	1200, 1200	"EASY"
TC-DIFF-02a	Average just above EASY boundary (avg=1201)	1202, 1200	"MEDIUM"
TC-DIFF-02b	Average exactly at MEDIUM boundary (avg=1500)	1500, 1500	"MEDIUM"
TC-DIFF-03a	Average just above MEDIUM boundary (avg=1501)	1502, 1500	"HARD"
TC-DIFF-03b	High average (avg=2000)	2000, 2000	"HARD"

2.4 RatingService – Elo Calculation (calculateRatingChange)

Formula: $newElo = \max(0, \text{round}(elo + K \times (score - expected)))$ where $K = 32$,

$$expected = \frac{1}{1 + 10^{(opponentElo - elo) / 400}}.$$

Return value is $newElo_1 - elo_1$.

ID	Description	elo1	elo2	Result	Expected delta
TC-RS-01	Equal Elo, winner	1000	1000	1 (win)	+16
TC-RS-02	Equal Elo, loser	1000	1000	-1 (loss)	-16

ID	Description	elo1	elo2	Result	Expected delta
TC-RS-03	Equal Elo, draw	1000	1000	0 (draw)	0
TC-RS-04	Rating floor – very low Elo loss	10	10	−1 (loss)	−10 (clamped to 0)
TC-RS-05	Favourite wins – small gain	1200	800	1 (win)	+3

Derivation for TC-RS-04: $newElo = \max(0, \text{round}(10 + 32 \times (0 - 0.5))) = \max(0, \text{round}(-6)) = 0$. Delta = $0 - 10 = -10$.

Derivation for TC-RS-05: $expected = 1/(1 + 10^{-1}) \approx 0.9091$. Delta = $\text{round}(32 \times (1.0 - 0.9091)) = \text{round}(2.909) = 3$.

2.5 ProblemController – HTTP Authorization Boundaries

ID	Description	Request	Expected HTTP Status
TC-PC-01	No Authorization header	POST /problem/createrequest (no header)	401 Unauthorized
TC-PC-02	USER role on admin endpoint	PUT /problem/setapproved/X with USER role JWT	403 Forbidden
TC-PC-03	Valid ADMIN token, no filters	GET /problem with ADMIN role JWT	200 OK

2.6 JudgeService – Code Execution and Verdict Evaluation

ID	Description	Input	Expected Output
TC-SUB-01	All test cases accepted → ACCEPTED verdict, <code>isWin()=true</code>	<code>evaluate("print(3)", "PYTHON", cases)</code> with Mockito stub returning "Accepted"	<code>verdict == "ACCEPTED", testsPassed == 2, isWin() == true</code>
TC-SUB-02	First test case fails with Wrong Answer → stops, 0 tests passed	<code>evaluate("print(0)", "PYTHON", cases)</code> with Mockito stub returning "Wrong Answer"	<code>verdict == "WRONG_ANSWER", testsPassed == 0, isWin() == false</code>

ID	Description	Input	Expected Output
TC-SUB-03	Time Limit Exceeded maps to TIME_LIMIT (not RUN-TIME_ERROR)	<code>evaluate("while True: pass", "PYTHON", cases)</code> with Mockito stub returning "Time Limit Exceeded"	<code>verdict == "TIME_LIMIT", isWin() == false</code>
TC-SUB-04	Compilation Error maps to COMPI-LATION_ERROR	<code>evaluate("def broken(:", "PYTHON", cases)</code> with Mockito stub returning "Compilation Error"	<code>verdict == "COMPILATION_ERROR", testsPassed == 0</code>
TC-SUB-05	First test passes, second fails → 1 test passed, not a win	<code>evaluate(...)</code> with Mockito returning "Accepted" (test 1) and "Wrong Answer" (test 2)	<code>verdict == "WRONG_ANSWER", testsPassed == 1, isWin() == false, failedTestNumber == 2</code>

3 Test Execution

3.1 Test Files and Configurations

File / Configuration	Location	Type
<code>JwtUtilTest.java</code>	<code>Backend/src/test/../../demo/</code>	Unit
<code>MatchmakingTest.java</code>	<code>Backend/src/test/../../demo/</code>	Unit
<code>RatingServiceTest.java</code>	<code>Backend/src/test/../../demo/</code>	Unit
<code>ProblemControllerTest.java</code>	<code>Backend/src/test/../../demo/</code>	Integration (MockMvc)
<code>JudgeServiceTest.java</code>	<code>Backend/src/test/../../demo/services/</code>	Unit
<code>eslint.config.js</code>	<code>frontend/</code>	Static analysis config

3.2 Execution Commands

```
# Backend unit & integration test suites:
cd Backend
./mvnw test

# Frontend static analysis & lint checks:
cd ../frontend
```



```
npm run lint
```

3.3 Execution Log (May 29, 2026)

```
[INFO] Running com.example.demo.JwtUtilTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.360 s -- in com.example.demo.JwtUtilTest
[INFO] Running com.example.demo.MatchmakingTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.875 s -- in
      com.example.demo.MatchmakingTest
[INFO] Running com.example.demo.ProblemControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.816 s -- in
      com.example.demo.ProblemControllerTest
[INFO] Running com.example.demo.RatingServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.063 s -- in
      com.example.demo.RatingServiceTest
[INFO] Running com.example.demo.services.JudgeServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.199 s -- in
      com.example.demo.services.JudgeServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 26, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.871 s

# Frontend static analysis & lint checks:
> frontend@0.0.0 lint
> eslint .

ESLint: No lint errors or warnings found.
```

4 Bug Reports

No failures were observed during automated test execution. The following issues were identified and fixed during development:

BR-01 – Type inference error in ProblemControllerTest

Severity: Build error (blocked compilation)
Component: ProblemControllerTest.java, TC-PC-03
Description: `when(problemService.getProblems(...)).thenReturn(ResponseEntity.ok(...))` failed to compile because `ProblemService.getProblems()` returns `ResponseEntity<?>` (unbounded wildcard). The Java compiler could not reconcile the wildcard type with the concrete `Map<String, List<Object>>` argument.
Fix: Replaced `when(...).thenReturn(...)` with `doReturn(...).when(problemService).getProblems(...)`. The `doReturn` form bypasses type inference and directly stubs the return value.
Status: Fixed. Test compiles and passes.

BR-02 – Invalid HTTP status code on token expiry in UserController

Severity: Medium (functional boundary error)
Component: UserController.java, line 102
Description: When a client request carries an expired or invalid JWT authorization token, the endpoint `/user/me` returned `ResponseEntity.status(200)` (OK) with a message body `{"result": "Your Authorization expired"}`. Since the status code was 200, the client-side axios handler processed the response as a successful load instead of redirecting the user to `/login`.
Fix: Updated the return status code from 200 to 401 (`ResponseEntity.status(401).body(...)`).
Status: Fixed. Verified by manual verification and token-expiry simulation.

BR-03 – Compilation failure due to missing SubmissionResult record

Severity: Build error (blocked compilation)
Component: JudgeService.java
Description: The backend failed to compile because `JudgeService` references the `SubmissionResult` class, which was omitted from the initial commit.
Fix: Implemented and added the `SubmissionResult` Java record containing `verdict`, `testsPassed`, `totalTests`, and `description` fields.
Status: Fixed. Verified backend compiles and builds cleanly.

5 Test Results

Test ID	Test Name	Suite	Result
TC-JWT-01	Username claim present	JwtUtilTest	PASS
TC-JWT-02	Role claim present	JwtUtilTest	PASS
TC-JWT-03	Subject equals UUID	JwtUtilTest	PASS
TC-JWT-04	Tampered token rejected	JwtUtilTest	PASS
TC-JWT-05	Invalid string rejected	JwtUtilTest	PASS
TC-MM-01	Same Elo always matches	MatchmakingTest	PASS
TC-MM-02	Diff=50 at tolerance (boundary)	MatchmakingTest	PASS
TC-MM-03	Diff=51 above tolerance (boundary)	MatchmakingTest	PASS
TC-MM-04	Both waited 35 s, diff=100	MatchmakingTest	PASS
TC-MM-05	Effective tolerance is minimum	MatchmakingTest	PASS
TC-DIFF-01	avg≤1200 → EASY (2 sub-cases)	MatchmakingTest	PASS
TC-DIFF-02	1200<avg≤1500 → MEDIUM (2 sub-cases)	MatchmakingTest	PASS
TC-DIFF-03	avg>1500 → HARD (2 sub-cases)	MatchmakingTest	PASS
TC-RS-01	Equal Elo win → +16	RatingServiceTest	PASS
TC-RS-02	Equal Elo loss → -16	RatingServiceTest	PASS
TC-RS-03	Equal Elo draw → 0	RatingServiceTest	PASS
TC-RS-04	Rating floor → DB update = 0	RatingServiceTest	PASS
TC-RS-05	Favourite win → +3	RatingServiceTest	PASS

Test ID	Test Name	Suite	Result
TC-PC-01	No token → 401	ProblemControllerTest	PASS
TC-PC-02	USER role on admin endpoint → 403	ProblemControllerTest	PASS
TC-PC-03	ADMIN token, no filters → 200	ProblemControllerTest	PASS
TC-SUB-01	All test cases accepted	JudgeServiceTest	PASS
TC-SUB-02	First test case fails	JudgeServiceTest	PASS
TC-SUB-03	Time Limit Exceeded maps	JudgeServiceTest	PASS
TC-SUB-04	Compilation Error maps	JudgeServiceTest	PASS
TC-SUB-05	First passes, second fails	JudgeServiceTest	PASS
TC-LINT-01	ESLint static compliance	Static Analysis (npm run lint)	PASS

Total: 27 tests – 27 PASS, 0 FAIL, 0 ERROR

6 Evaluation

6.1 Coverage Analysis

Component	Logic Covered	Logic Excluded
JwtUtil	Token creation, claim extraction, signature verification, invalid input	Token expiry (would require mocking <code>System.currentTimeMillis</code>)
MatchmakingDto. <code>canMatchWith</code>	All branches: match, reject (boundary), tolerance growth, min-tolerance rule	None – full branch coverage achieved
MatchmakingService. <code>calculateDifficulty</code>	All three outcome branches with boundary values	None – full branch coverage achieved
RatingService. <code>calculateRatingChange</code>	Win/loss/draw paths, K-factor application, floor clamp, DB write verification	Very low-elo upsets (floor triggered differently for very asymmetric ratings)
ProblemController (HTTP layer)	Missing token, wrong role, correct role + delegation	Missing token on other endpoints (same code path, redundant)

Component	Logic Covered	Logic Excluded
JudgeService	Verdict mapping (AC, WA, TLE, CE), early termination, partial pass/failed test index tracking	External live network HTTP requests to the Judge0 REST API (mocked out)
MatchHandler / MatchmakingHandler	Not covered – see §1	WebSocket session lifecycle, concurrent queue operations
UserService	Not covered – login/register flow exercises BCryptPasswordEncoder and UserRepository; tested manually	
React Frontend	Static analysis checks (ESLint) over all components and service hooks	Dynamic UI interactions and browser rendering
Repository SQL	Not covered – see §1	All SQL queries

6.2 What the Tests Do Not Cover

The most significant gap is the **concurrent matchmaking queue**. `MatchmakingService.processMatchmakingQueue` is a `@Scheduled` method that manipulates a `ConcurrentLinkedQueue` under a `synchronized` block. Race conditions between the scheduler thread and player-disconnect callbacks are not tested. No automated test reproduces a scenario where two scheduler ticks run close together or a player is removed mid-match.

The **Judge0 integration path** inside `MatchService` is similarly untested. The code sends HTTP requests to `localhost:2358` synchronously; no mock server or stub is in place.

7 Test Summary Report

All 26 automated tests pass. The one compilation-level defect found (BR-01, type inference) was fixed before the suite ran.

The automated tests achieve full branch coverage over the four pure-logic components (`JwtUtil`, `MatchmakingDto`, `MatchmakingService.calculateDifficulty`, `JudgeService`) and confirm the Elo formula and rating floor in `RatingService`. The controller integration tests confirm that the authentication and role-check guard clauses work end-to-end at the HTTP layer for the three representative cases.

The principal untested areas – WebSocket handlers and the repository SQL layer – are accepted as out of scope for automated testing within this semester project, consistent

with the decision log entries below. Both areas were verified by manual end-to-end testing during development.

The system is considered ready for project submission and demo.

8 Decision Log – Testing Strategy

8.1 DS-01 – Use Mockito for repository isolation rather than a test database

Decision:	Repository interfaces are mocked with Mockito in unit tests. No in-memory or containerised database is used.
Alternatives considered:	(1) H2 in-memory database. (2) Testcontainers with a real PostgreSQL instance.
Rationale:	The repository implementations use Spring Data JDBC with SQL that may include PostgreSQL-specific syntax (e.g. UUID casting). H2 would reject these statements without significant compatibility shims. Testcontainers would require Docker and a CI-ready environment neither of which is guaranteed in the course lab. Mocking the repositories is sufficient to test the service logic in isolation.
Trade-offs:	SQL correctness is not verified by automated tests. A repository bug (wrong column name, wrong JOIN) would not be caught at unit level.
Risks / technical debt:	If repository methods are refactored, the mocks silently continue to return the configured values; divergence between mock behaviour and real SQL is possible.

8.2 DS-02 – Exclude WebSocket handlers from automated tests

Decision:	MatchHandler and MatchmakingHandler are not covered by any automated test.
Alternatives considered:	Spring's WebSocketStompClient for integration tests; a custom StandaloneWebSocketHandlerAdapter setup for unit tests.
Rationale:	Both approaches require significant setup for a live WebSocket session (including authentication handshake, session lifecycle, and timing of @Scheduled tasks). The complexity-to-benefit ratio is too high within the one-semester project timeline. Manual demo covers these flows adequately for the scope of this course project.
Trade-offs:	Concurrency bugs in processMatches() (race conditions, queue re-insertion order) remain undetected by automated tests.
Risks / technical debt:	A subtle timing bug in the matchmaking scheduler might only appear under load (multiple simultaneous players). This risk is accepted.

8.3 DS-03 – Use MockMvc standaloneSetup instead of @WebMvcTest for the controller integration test

Decision:	<code>ProblemControllerTest</code> uses <code>MockMvcBuilders.standaloneSetup(new ProblemController(...))</code> instead of the <code>@WebMvcTest</code> slice annotation.
Alternatives considered:	<code>@WebMvcTest(ProblemController.class)</code> with <code>@MockBean;</code> <code>@SpringBootTest</code> with <code>@AutoConfigureMockMvc.</code>
Rationale:	<code>@WebMvcTest</code> and <code>@MockBean</code> require <code>spring-boot-test-autoconfigure</code> which is not bundled with the <code>spring-boot-starter-webmvc-test</code> artifact used in this project's <code>pom.xml</code> . <code>@SpringBootTest</code> would attempt to load the full application context including a live database connection. <code>standaloneSetup</code> loads only the controller under test and exercises the same request-dispatch path as a full MVC test.
Trade-offs:	<code>standaloneSetup</code> bypasses Spring's <code>DispatcherServlet</code> exception resolvers; exceptions not caught by the controller itself would not be translated to HTTP responses. For this controller, all error cases are explicitly handled with <code>ResponseEntity</code> returns, so this is not a concern.
Risks / technical debt:	If a <code>@ControllerAdvice</code> global exception handler is added later, it would not be exercised by this test.